Python in the AI Era: A Crash Course

Mode 2 Phase 1 (Building Blocks), Chapter 1/6 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/python-crash-course

EN: You now READ Python, the agent WRITES it -- 10-80-10 applied: you own what to build and verify, the agent generates the middle 80%. The skill is recognition, not recall.
Roman Urdu: Ab aap Python PARHTE ho, agent LIKHTA hai -- 10-80-10 applied: aap kya banana/verify karna hai own karte, agent beech ke 80% generate karta. Skill recognition hai, recall nahi.

1. PRIMM-AI+ + the Workbench

- **5-step method:** Predict (guess before running) -> Run -> Investigate (guess vs reality) -> Modify (change one thing) -> Make (specify your own, agent builds, you verify).
- Workbench tools are 'reading machines' that read agent-generated code for you: uv, pytest, Pyright (type-check), Ruff (style).

2. Shapes You See Everywhere

- Function signature = a CONTRACT -- the body can be wrong, the signature states the promise. Dict = JSON's direct Python map, the main way agents pass data.
- Verify-worthy bug spots: non-terminating while, off-by-one slicing, silently-swallowing except, unconverted input().

3. AI-Era Power Concepts (Read-Only Zone)

- Type hints steer the agent. Pydantic validates at RUNTIME with clear errors -- used everywhere in agent code because it auto-generates JSON schemas for tool calling.
- Generators (yield) stream one item at a time instead of loading a whole list. async/await: 20 sequential API calls ~2s each vs. async ~1 slowest-call's time.

4. Test-Driven Generation (TDG)

- Order reversed from old TDD: YOU write the failing test that defines 'correct' (never the agent -- or it's not an independent signal), agent generates the implementation, test verifies.
- If the agent changes the TEST to pass (not the code), stop it. Read tracebacks bottom-up.
- Decomposition: goal in one sentence -> list steps (verbs = candidate units) -> typed signature per unit -> TDG each unit in order -> wire together. Heuristic: if a test is hard to write, the unit is doing too much.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
PRIMM-AI+	Predict, Run, Investigate, Modify, Make -- the course's testing method.	Predict, Run, Investigate, Modify, Make -- course ka testing method.
Test-Driven Generation (TDG)	You write the failing test first; the agent generates code to pass it.	Aap pehle failing test likhte; agent us ko pass karne ke liye code generate karta.
Type hints	Labels that steer what the agent generates.	Labels jo decide karte agent kya generate kare.
Pydantic	Runtime validation with clear errors, also used to auto-generate tool-calling JSON schemas.	Runtime validation, clear errors ke sath; tool-calling JSON schemas auto-generate karta.
Diff review checklist	7 points checked before accepting agent-written code.	7 points jo agent-written code accept karne se pehle check hote.

Quick Self-Check

Question	Answer
What does the 10-80-10 rule mean in this Python course specifically?	You own the first 10% (define correct) and last 10% (verify); the agent generates the middle 80% of code.
Why must YOU write the TDG test, never the agent?	If the agent writes its own test, it's no longer an independent signal -- it could write a test that matches whatever it happens to generate.
What's the red flag if an agent 'fixes' a failing test?	If it changed the TEST instead of the code to make it pass, that's a sign to stop and investigate, not accept the fix.
Why are dunder methods like __call__ something to recognize, not write?	This whole course is about reading/verifying agent-generated code, not authoring these patterns yourself.

Where This Leads / Sources

Leads to: Build AI Agents, Postgres for AI, Building a Digital FTE. Full 6 practice projects + 60-second glossary in 04.

Source: docs/python-crash-course/ (this repo), content originally via Zia Tutor AI connector.